MID-PROJECT REPORT

C++ Report

Data Structures and 3D Rendering

Prepared by

Raïan CHOUIREF

Dorian LEON FERNANDES

Academic Year: 2026–2027

October 16, 2026

Contents

1 Introduction 1

2 Problem Statement 1

3 Project Context 2

4 UML Architecture 3

5 Loader 6

6 Halfedge Data Structure 7

7 Rendering 8

8 Conclusion 9

1. Introduction

This project consists of developing a 3D editor capable of loading, displaying, and manipulating 3D models from files in .PLY and .OBJ formats. The main objective is to allow the user to interact with 3D models intuitively and efficiently. To achieve this, we need to implement a suitable data structure to represent 3D meshes, as well as a high-performance rendering system using OpenGL. The major steps of the project include designing the Halfedge data structure, developing the file loader, implementing the rendering system, and creating a user interface.

2. Problem Statement

In this project, we address the challenge of manipulating and rendering 3D models from .PLY and .OBJ files. The primary goal is to build a 3D editor that can load these files, display the corresponding models, and enable user interaction. The specific objectives of the project include:

- Setting up an efficient data structure to represent 3D meshes, specifically using the Halfedge structure.
- Developing a rendering system using OpenGL to display 3D models realistically and efficiently.
- Creating a file loader capable of reading .PLY and .OBJ formats, extracting geometric information, and converting it into data structures usable by the program.

3. Project Context

As part of the Computer Science course, we were tasked with carrying out a C++ project focused on programming and geometry. The main objective was to develop a 3D editor capable of loading, displaying, and manipulating 3D models from .PLY and .OBJ files. To accomplish this, we used several tools and libraries, notably OpenGL for graphical rendering, alongside tailored data structures to represent 3D meshes. The project constraints included managing geometric data efficiently, providing an intuitive user interface, and ensuring compatibility with various 3D file formats. We also had to ensure that our editor could handle complex models while maintaining optimal performance.

So far, we have successfully implemented basic features, including loading .PLY and .OBJ files and rendering 3D models. However, further improvements are still needed, particularly regarding performance optimization and adding extra features to enrich the user experience.

4. UML Architecture

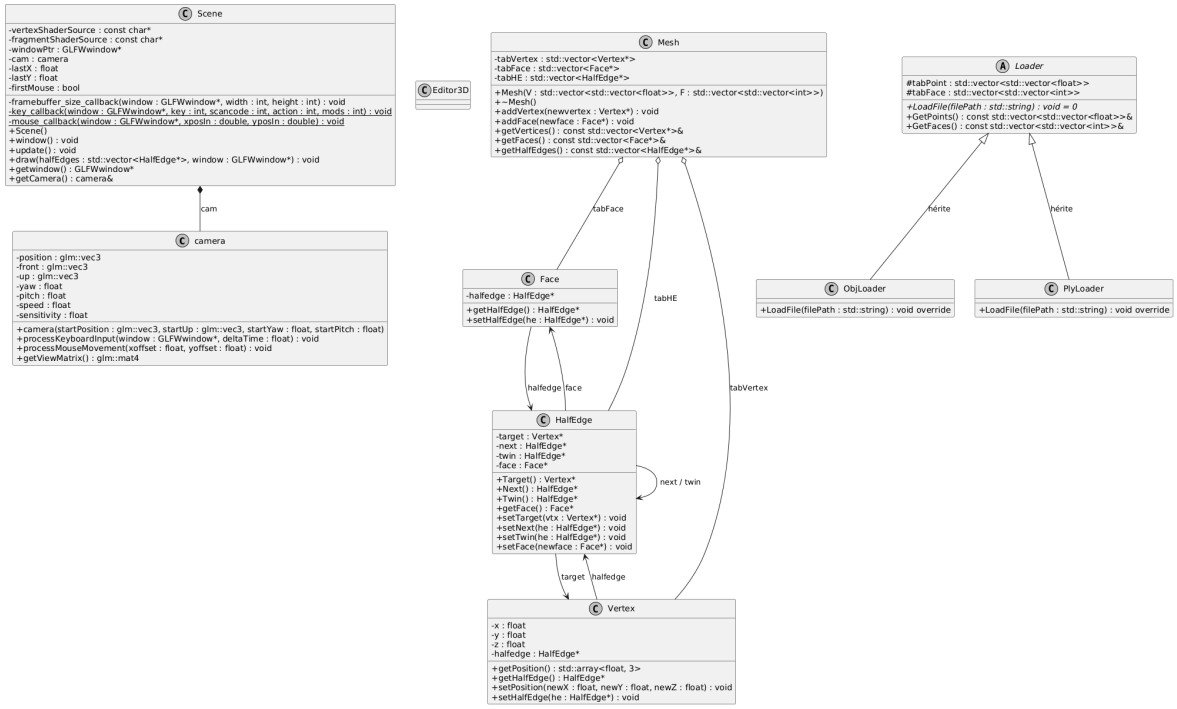


Figure 1: Project UML Diagram

We used a UML diagram to represent the classes and their relationships within our project. The diagram illustrates the main classes, their attributes and methods, as well as their associations. This helped us better understand the project architecture and facilitated communication among team members.

Here is an overview of the main classes and their relationships:

- The `Scene` class represents the 3D scene and contains objects such as meshes and cameras. It handles rendering management and user interaction with objects.
- The `Mesh` class represents a 3D mesh and stores information about vertices, faces, and edges. It manages geometric data and generates data structures required for rendering.
- The `Halfedge` class represents a data structure for meshes, enabling efficient navigation between vertices, edges, and faces. It facilitates mesh modification and rendering operations.

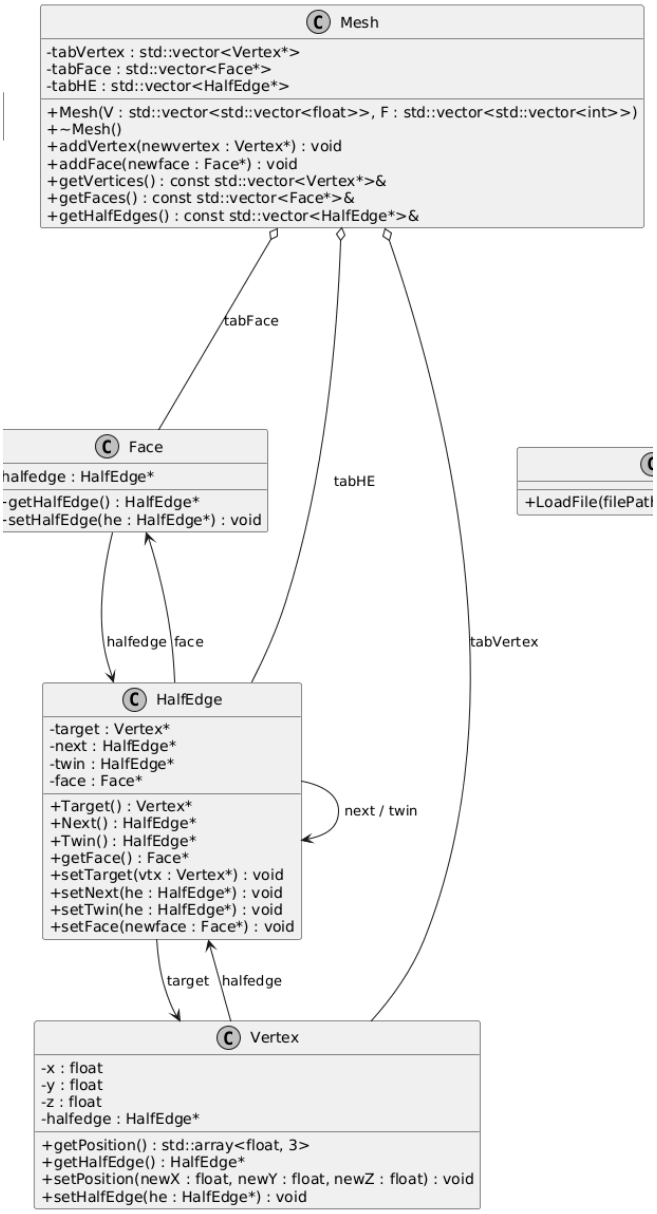


Figure 2: UML Diagram of the Mesh Structure

This section shows the UML diagram of the `Mesh` structure, representing a 3D mesh. The `Mesh` class contains attributes to store vertices, faces, and edges, along with methods to manage them. Relationships between classes are also shown, demonstrating how they interact to form the overall project architecture.

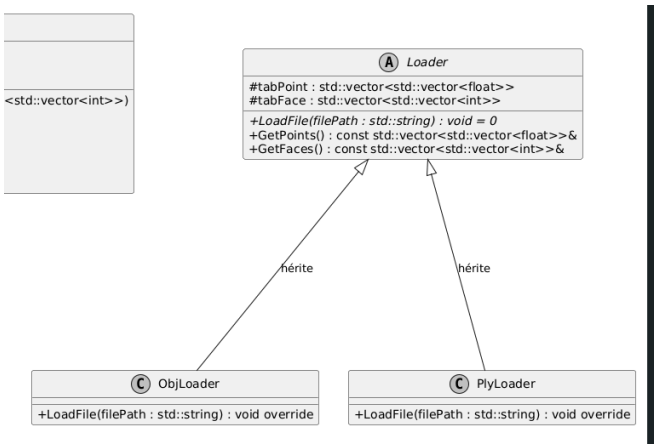


Figure 3: UML Diagram of the Loader

For the loader, we used a generic class design to support loading multiple file formats. This allowed us to create specific classes for each file format, such as `PLYLoader` and `OBJLoader`, which inherit from the generic `Loader` class. This architecture makes it straightforward to add new file formats in the future.

5. Loader

The file loading system in our project relies on two specific classes, `PlyLoader` and `ObjLoader`, both inheriting from the generic `Loader` class.

The `PlyLoader` class loads `.ply` files encoded in either ASCII or Little Endian binary format. It begins by reading the header to determine the file format, vertex count, face count, data types used (`double`, `float`, `int`), and potential extra properties like color or opacity. Implementing this loader required handling bytes to be skipped during binary reading, such as padding or unused properties. We leveraged AI tools like ChatGPT to implement this logic and discover suitable C++ utilities like `std::istringstream`, which simplifies parsing ASCII data.

The `ObjLoader` class handles `.obj` files. It parses each line based on its prefix to distinguish vertices (identified by `v`) from faces (identified by `f`). Using `std::istringstream` allows for clean extraction of the values on each line.

Finally, both classes store extracted data into two main arrays: `tabPoint` and `tabFace`. The first contains vertex coordinates as a `std::vector<Eigen::Vector3d>`, while the second contains faces as a `std::vector<std::vector<int»`. This organization represents vertices and connectivity in structures directly usable by the rest of the application.

6. Halfedge Data Structure

Our Halfedge structure relies on three closely related classes: `Vertex`, `Face`, and `HalfEdge`.

The `Vertex` class represents mesh vertices. Each vertex stores a position as an `Eigen::Vector3d` and a pointer to an associated `HalfEdge*`.

The `Face` class represents triangular mesh faces. It contains a pointer to one of its constituent `HalfEdge` objects (`HalfEdge*`). Other halfedges belonging to the face can then be traversed via their internal links.

The `HalfEdge` class is the core of this data structure. Each halfedge contains several pointers describing its position within the mesh: a pointer to its target vertex (`Vertex*`), a pointer to the next halfedge (`next`), a pointer to its opposite twin halfedge (`twin`), and a pointer to its parent face (`Face*`).

These links allow efficient traversal of the mesh and quick access to neighboring elements. They are designed to facilitate geometric and topological modifications, though these editing features are not yet fully implemented.

The mesh structure is generated by the `Mesh` class using the vertex and face arrays provided by the loader. Elements are then organized into three arrays: `tabVertex`, `tabFace`, and `tabHE`, storing vertices, faces, and halfedges respectively.

7. Rendering

For rendering, we used the `OpenGL` library, a powerful and flexible graphics API for 2D and 3D rendering. OpenGL allows us to create interactive 3D scenes and manage transformations, lighting, and textures.

Below are the vertices of a cube used to test rendering in our 3D editor:



Figure 4: Cube vertices for rendering tests

And the corresponding triangular faces:

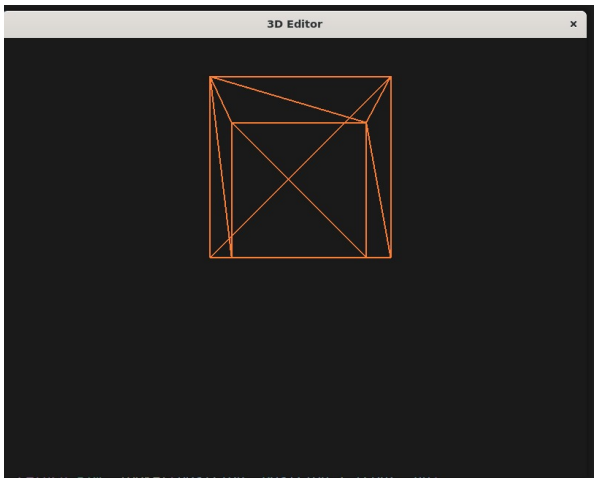


Figure 5: Cube triangular faces for rendering tests

Regarding camera navigation, only WASD/ZQSD movement is currently implemented. Camera displacement is achieved by updating its position in 3D space based on user input. We also plan to add extra features such as zooming and camera rotation to improve the user experience.

8. Conclusion

Most of the structural foundations of the project have been successfully established. However, several features still need to be developed to reach the final objectives.

The primary difficulty encountered was our limited prior experience with C++, particularly regarding OpenGL and the various utility libraries. This required extra time to understand how they work and how to use them properly. Upcoming tasks include implementing mesh geometry editing, shortest path computation between arbitrary vertices, performance optimization, and robust error handling to prevent crashes.